CYBERSENTINEL COPILOT

Module-wise architecture with aligned Synapt agent flow

Runtime order on top · owning Python modules below · Synapt roles aligned to their consuming stages

A · PRIMARY INVESTIGATION FLOW — MODULE ORDER

1 · API entry
Validate contract
Create graph state
Dispatch request

2 · Guardrail
Injection/action check
Domain relevance
Pass or reject

3 · Flow triage
Load exact flow / 404
Hybrid incident RAG
Self-exclusion

4 · ML analysis
Classify + risk
Feature deviations
Anomaly + MITRE

5 · Knowledge
Sanitize context
Retrieve procedures
Provider fallback

6 · Priority
Apply P1–P4 policy
Rationale
Missing context

7 · Response
Build steps/actions
Provenance
Limitations

8 · Judge
Hard safety gate
Four-part rubric
Critique/verdict

9 · UI + analyst
Render evidence
Human decision
Submit feedback

B · OWNING PYTHON MODULES — ONE COLUMN PER RUNTIME STAGE

Module owner
main.py
schemas.py

Module owner
guardrails.py

Module owner
sqlite_store.py
rag/historical_rag.py

Module owner
ml/predictor.py
ml/anomaly.py

Module owner
workflow.py
synapt/client.py

Module owner
policy.py

Module owner
workflow.py
response builder

Module owner
judge/evaluator.py
evaluation adapter

Module owner
ui/index.html
feedback/improvement.py

C · SYNAPT AGENT ALIGNMENT AND PHYSICAL PROVIDER FLOW

Historical Incident RAG Agent
historical-incident-rag-agent
Similarity over 291 summaries
Declared: search + query

Threat Intel / SOP Agent
incident-knowledge-agent
NIST · MITRE · procedures
Declared: query + procedures

Judge Review Agent
judge-review-agent
Safety + grounding rubric
Declared: query

Shared SynaptClient runtime
TLS/authentication → MCP initialize → tools/list → tools/call · retrieve() prefers trident_query and falls back to trident_search

Synapt Context Substrate · provider assist_cyber
Shared knowledge, semantic retrieval, and judge response boundary

Alignment rule: the three names are logical dashboard roles; current code does not choose a separate credential/client per role.

D · DATA, MODEL, POLICY, AND STATE MODULES

SQLite evidence
2,313,810 flows
Exact lookup + structured filters
Source-file/row provenance

Historical retrieval state
291 incident summaries
FTS5 BM25 index
RRF-ranked response

Local ML artifacts
65-feature classifier · 9 families
IsolationForest + threshold
Benign feature baseline

Policy and mappings
P1–P4 deterministic rules
MITRE technique map
Static response template

Feedback and reports
analyst_feedback
Candidate model bundle
Benchmark/evaluation JSON

E · SUPPORTING ENDPOINT MODULE FLOWS

Search module
POST /search/incidents
Store filters + HistoricalIncidentRAG
→ historical role → shared client

Chat module
POST /chat
Guardrails + store + predictor
→ knowledge role → shared client

Feedback module
POST /feedback and /review
SQLite human-review lifecycle
Optional direct trident_feedback

Improvement module
POST /feedback/improve
Accepted feedback + 2,000-row holdout
Candidate bundle + report

Status module
GET /health + /api/synapt/agents
SQLite readiness + model indicators
Logical agent configuration status

F · OFFLINE PREPARATION, EVALUATION, AND IMPROVEMENT

Dataset ingestion
Parquet audit → normalize labels/features
→ SQLite flows + indexes

Model training
SQLite sample → 65 features
→ classifier + anomaly bundles

Knowledge ingestion
MITRE/NIST/schema/summaries
→ trident_ingest → Synapt

Quality evaluation
40 fixtures + adapter + safety gate
→ DeepEval-labelled JSON report

Feedback benchmark
Reviewed corrections + active model
→ candidate and decision report

Implementation boundary

Synapt roles are aligned by purpose in the UI and architecture. Physically, HistoricalIncidentRAG, knowledge retrieval, judge evaluation, and chat all call the same SynaptClient.retrieve() path; per-agent routing is not implemented in the Python runtime.

Solid arrows: runtime order · dashed arrows: module/data dependency · containment remains a human decision